

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 05 -

KUBERNETES NA PRÁTICA: IMPLANTANDO MODELOS DE ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS.....	13

EMSE

O QUE VEM POR AÍ?

Agora que você já compreende os fundamentos da orquestração, esta aula foca na implantação prática de modelos de ML em Kubernetes.

O objetivo é transformar uma aplicação de inferência containerizada em um serviço estável, com manifests claros, estratégia de exposição e comportamento previsível em produção.

Você vai aprofundar o uso de Deployments e Services em cenários reais de inferência síncrona, entendendo como decisões de configuração afetam latência, disponibilidade e capacidade de evolução do serviço.

HANDS ON

No hands on, você acompanhará o deploy de uma API de inferência no cluster, desde a aplicação do primeiro manifesto até a validação de chamadas reais ao endpoint. As videoaulas detalham a leitura de status dos objetos e a interpretação de sinais operacionais durante a implantação.

Durante a prática, serão ajustados parâmetros de réplica, recursos e probes para mostrar como esses elementos influenciam prontidão, estabilidade e experiência do consumidor da API. O foco é construir um deploy que não apenas suba, mas que se mantenha saudável em condições próximas do uso real.

Ao final, você terá um roteiro técnico reutilizável para publicar modelos de forma mais segura e consistente, com base em critérios mensuráveis e boas práticas aplicáveis ao dia a dia de MLOps.

SAIBA MAIS

Implantação de modelos: artefato técnico

Implantar um modelo em Kubernetes significa assumir compromisso de produção. O objetivo não é apenas responder requisições, mas fazer isso com latência previsível, disponibilidade consistente e comportamento auditável ao longo de múltiplas versões.

Por isso, o manifesto de deploy deve representar claramente o contrato operacional do serviço: como inicializa, como escala, como expõe endpoint e como se recupera de falhas.

Antes de aplicar manifests, confirme se:

- A imagem versionada é reproduzível.
- O endpoint de saúde foi implementado.
- Os logs estão estruturados.
- A configuração externa está pronta para ambiente.

Se esses pré-requisitos não existirem, o Kubernetes apenas ampliará os problemas em escala.

Deployment com visão de produção

O deployment gerencia réplicas e rollout. Para inferência síncrona, o número de réplicas deve considerar concorrência, perfil de tráfego e tempo médio de processamento.

Além disso, requests e limits precisam ser ajustados com dados reais. Subdimensionar cria saturação; superdimensionar eleva custo sem ganho proporcional.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: recomendacao-api
spec:
```

```
replicas: 3
selector:
  matchLabels:
    app: recomendacao-api
template:
  metadata:
    labels:
      app: recomendacao-api
  spec:
    containers:
      - name: api
        image: recomendacao-api:2.3.0
        ports:
          - containerPort: 8000
        resources:
          requests:
            cpu: "300m"
            memory: "512Mi"
          limits:
            cpu: "1"
            memory: "1Gi"
```

Código-fonte 1 – Exemplo de Deployment para inferência com recursos definidos (YAML)
Fonte: Elaborado pelo autor (2026)

Probes e o ciclo de warm-up do modelo

Modelos frequentemente exigem tempo de carga inicial. Readiness e liveness devem refletir esse comportamento real para evitar tráfego prematuro ou reinício desnecessário.

Além disso, eles servem como “verificador de estado” do seu pod, realizando o healthcheck e garantindo que não apenas o processo esteja em running, mas que a aplicação também esteja apta a receber chamadas.

Exemplo de probes para API de inferência com warm-up:

```
livenessProbe:
  httpGet:
    path: /health/live
    port: 8000
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8000
  initialDelaySeconds: 20
```

```
periodSeconds: 5
failureThreshold: 3
```

Código-fonte 2 – Exemplo de probes para API de inferência com warm-up
Fonte: Elaborado pelo autor (2026)

Exemplo 1 - readiness incorreta em produção: uma API recebia tráfego antes de concluir o carregamento do modelo, causando erros intermitentes no início do deploy. Após o ajuste da readiness probe com janela adequada, a taxa de erro de startup caiu drasticamente.

Estratégias de exposição e governança de entrada

A forma de exposição (ClusterIP, NodePort, LoadBalancer, Ingress) muda a arquitetura de acesso e segurança. Em ambientes corporativos, Ingress costuma centralizar TLS, roteamento e políticas.

```
apiVersion: v1
kind: Service
metadata:
  name: recomendacao-svc
spec:
  selector:
    app: recomendacao-api
  ports:
    - port: 80
      targetPort: 8000
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: recomendacao-ingress
spec:
  rules:
    - host: ml.exemplo.com
      http:
        paths:
          - path: /recomendacao
            pathType: Prefix
            backend:
              service:
                name: recomendacao-svc
                port:
                  number: 80
```

Código-fonte 3 – Exemplo de Service + Ingress para exposição controlada (YAML)
Fonte: Elaborado pelo autor (2026)

ESTRATÉGIA	VANTAGEM	LIMITAÇÃO	USO RECOMENDADO
ClusterIP	Simples e interno	Sem acesso externo direto	Comunicação entre serviços internos
NodePort	Rápido para teste	Menor governança	Ambientes de laboratório
LoadBalancer	Exposição direta	Custo e controle disperso	Cloud com baixa complexidade de roteamento
Ingress	Governança central	Configuração mais rica	Produção corporativa

Tabela 1 – Estratégias, vantagens, limitações e usos recomendados
Fonte: Elaborado pelo autor (2026)

O desempenho da inferência depende de toda a cadeia: rede, serialização, runtime, concorrência e recursos de nó. Focar só no tempo de predição do modelo é insuficiente para entender a experiência do usuário.

Por isso, testes de carga devem acompanhar deploy funcional, especialmente antes de promover para produção.

Publicar uma nova versão de modelo sem estratégia de rollout é aumentar risco. O rolling update é base útil, mas em cenários críticos canary ou blue/green podem reduzir o impacto.

Defina critérios explícitos de sucesso e gatilhos de rollback para evitar decisões subjetivas sob pressão.

Durante a implantação, observe:

- O estado do Deployment.

- Os eventos de Pod.
- O readiness/liveness.
- A latência e taxa de erro do endpoint.

Essa leitura em tempo real evita que problemas avancem silenciosamente.

Exemplo - rollout progressivo com mitigação: uma nova versão elevou o consumo em 90% em horário de pico. Como o rollout era progressivo, o time interrompeu a promoção e voltou para a versão anterior sem indisponibilidade global. O aprendizado foi incorporado no checklist de validação pré-release.

NÍVEL	CARACTERÍSTICA	RISCO PRINCIPAL	PRÓXIMO PASSO
Inicial	Fragilidade sob carga	Sem acesso externo direto	Ajustar recursos e probes
Intermediário	Deploy com observabilidade inicial	Menor governança Rollback lento	Formalizar estratégia de rollout
Avançado	Deploy governado por métricas	Custo e controle disperso	Complexidade de operação

Tabela 1 - Tabela comparativa de maturidade de implantação
Fonte: Elaborado pelo autor (2026)

Erros comuns de adoção e mitigação

- Erro comum 1: deploy sem requests/limits.
Correção: calibrar com testes de carga reais.
- Erro comum 2: probes padrão sem aderência ao modelo.
Correção: medir warm-up e ajustar thresholds.

Erro comum 3: exposição sem política de segurança.

Correção: padronizar entrada com TLS e controle de acesso.

Implantar modelos em Kubernetes com qualidade exige juntar design de aplicação, configuração de plataforma e operação orientada por métricas. Esse conjunto transforma “subir serviço” em “sustentar serviço” com confiabilidade de produção.

EMENDAS

MERCADO, CASES E TENDÊNCIAS

Empresas que operam múltiplos serviços de dados vêm adotando padrões de deploy em Kubernetes para reduzir tempo de entrada em produção e melhorar previsibilidade de release. Esse padrão é especialmente importante em ML, onde novas versões de modelo são frequentes e exigem controle fino de rollout.

A tendência é ampliar essa abordagem com estratégias progressivas de tráfego (como canary e blue/green) para reduzir risco em publicações de modelos com alto impacto de negócio.

Um caso de modernização com Kubernetes é o do jornal The New York Times, que relata ganhos de agilidade e eficiência ao migrar cargas para arquitetura baseada em containers e orquestração e trata de milhões de acessos ao mês.

Outra tendência voltada é o uso de arquitetura Cloud Native onde nossas aplicações já nascem pensadas para rodar nos ambientes que estamos estudando e com as premissas de observabilidade, resiliência, escalabilidade e reprodutibilidade.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu a implantar modelos de ML em Kubernetes com Deployments e Services, entendendo como estruturar exposição, estabilidade e operação de inferência síncrona com foco em produção.

EMENDAS

REFERÊNCIAS

BURNS, B.; BEDA, J.; HIGHTOWER, K. **Kubernetes: Up and Running**. Sebastopol: O'Reilly Media, 2022

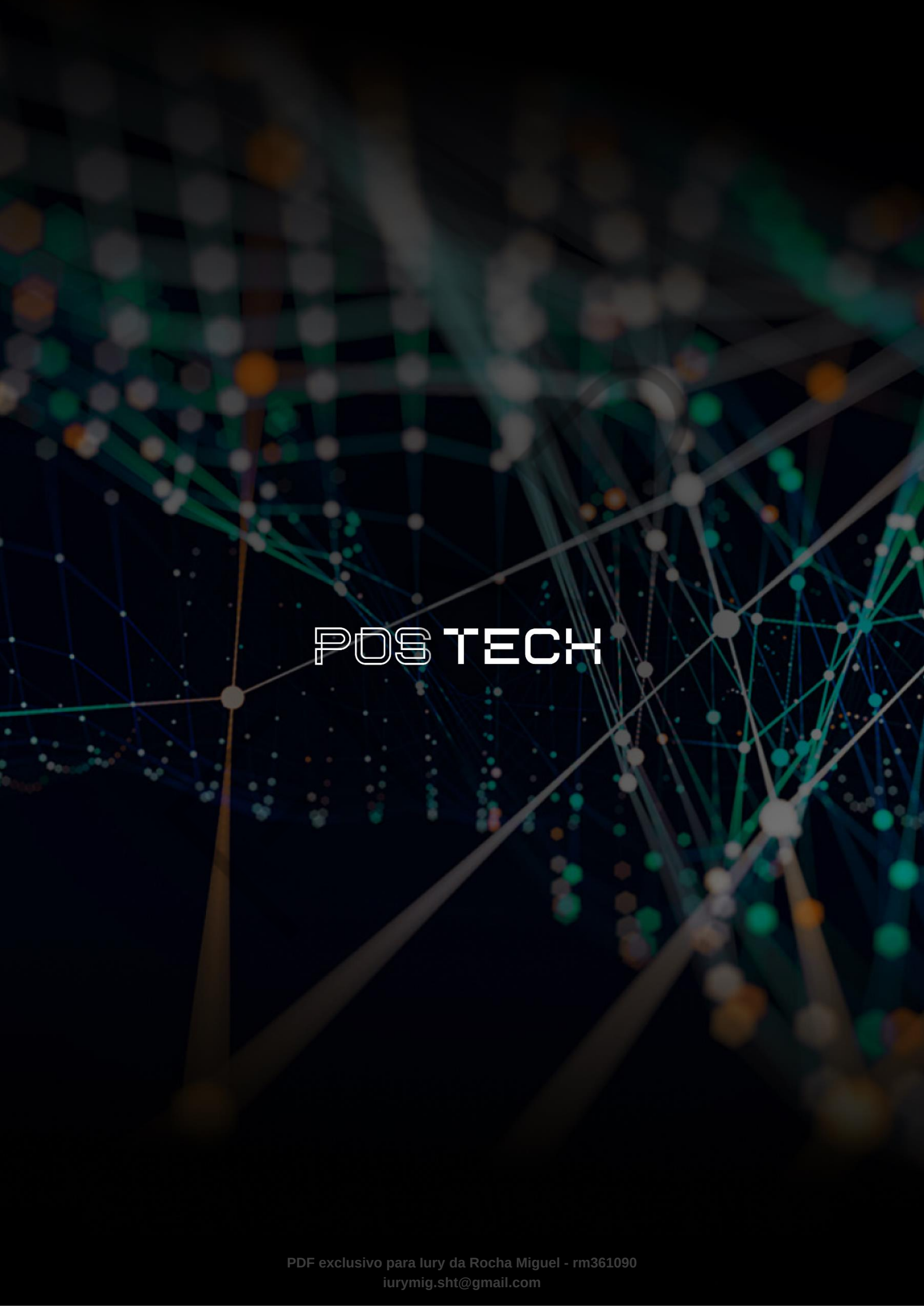
MORGAN, L.; ARUNDEL, J. **Cloud Native DevOps with Kubernetes**. Sebastopol: O'Reilly Media, 2022.

SATO, D. **DevOps na prática**. São Paulo: Casa do Código, 2014.

PALAVRAS-CHAVE

DevOps. Containers. Docker. Kubernetes.

EMEND



POSTECH